

Claude Code

基础知识

- 快速开始
- 快捷键
- 斜杠命令

1. 在根目录启动 claude

启动 `claude` 时, 在你项目的根目录运行, Claude Code 可以看到整个项目的结构, 会工作的更好.

2. 立即运行 /init

创建 `CLAUDE.md` , 它是 Claude Code 的长期记忆.

3. CLAUDE.md 层级结构

全局记忆 `~/.claude/CLAUDE.md` , 项目记忆 `./CLAUDE.md` , 其他规则 `./.claude/rules/**/*.md`

4. 保持 CLAUDE.md 的简洁

上下文越臃肿, AI 就越不可能完全按照你想要的方式去做.

5. CLAUDE.md 的结构

What: 技术栈, Domain: 每部分是干什么的, Validation: 测试和类型检查.

6. SHIFT + TAB 切换模式

Normal(普通模式) AUTO-Accept(自动接受模式) Plan Mode(计划模式).

7. ESC 中断

如果你觉得 AI 走偏了, 立即打断它, 不要等到它犯错再去纠正.

8. 双击 ESC 清空输入框

双击 ESC 可以清空整个输入框的提示词.

9. 双击 ESC 触发 rewind

如果输入框是空的, 双击 ESC 会触发 `/rewind` , `rewind code` 或者 `rewind conversation`, 或者全部 `rewind`.

10. 图片识别

提供图片路径或者拖拽图片, 并增加额外的上下文让 claude 知道你主要关注什么.

12. /context

使用 `/context` 可视化展示当前会话的上下文使用情况.

13. /compact

使用 `/compact` 手动压缩上下文, 或者在 `~/.claude.json` 中设置 `autoCompactEnabled: true` 自动压缩上下文.

14. /resume

使用 `/resume` 回到会话, 恢复上下文.

15. /rename

使用 `/rename` 自定义会话名称, 快速找到会话.

15. /mcp

使用 `/mcp` 查看 MCP Server 的状态.

16. /help

使用 `/help` 展示所有的命令.

Claude Code

高级技巧

- 下载与更新
- 绕过登录
- 结合插件
- Explore `~/ .claude`
- Update `CLAUDE.md`
- Self-improving
- Agent Security

17. 下载与更新

使用 npm 管理 Claude Code.

```
# 设置镜像
npm config set registry https://registry.npmmirror.com -g

# 安装
npm install -g @anthropic-ai/claude-code

# 安装指定版本(常用于回退版本)
# claude-code 更新频率极快, 新版有 bug 就立即回滚
npm install -g @anthropic-ai/claude-code@2.1.51

# 更新 - npm
npm install -g @anthropic-ai/claude-code@latest

# 更新 - claude
claude update
```

18. 绕过登录

Claude Code 运行时会检查是否完成登录引导.

在 `~/.claude.json` 中手动增加:

```
{  
  // 是否已完成首次启动的引导流程, true 表示不再展示欢迎向导  
  "hasCompletedOnboarding": true,  
  // 上次完成引导时的版本号, 版本升级后可能触发新的引导提示  
  "lastOnboardingVersion": "1.0.59",  
  // 已显示订阅提示的次数, 用于控制提示频率, 避免反复打扰  
  "subscriptionNoticeCount": 0,  
  // 当前账户是否有有效订阅  
  "hasAvailableSubscription": false,  
  // 当可用 token 用量降到该阈值以下时触发警告, 0 表示禁用此警告  
  "fallbackAvailableWarningThreshold": 0  
}
```

19. 结合插件

Vscode, JetBrains IDEs 均包含.

20. Explore `~/.claude`

会话, 配置, 工具等, 所有的一切都在 `~/.claude` .

21. Update CLAUDE.md

当你纠正错误后, 让模型总结并更新 CLAUDE.md

22. Self-improving

Learning from sessions, Self-improving from reflections.

每次会话结束

↓ `/diary`

结构化日记文件 (`~/.claude/memory/diary/YYYY-MM-DD-session-N.md`)

- 任务摘要 / 完成工作 / 设计决策
- 用户偏好 / 代码风格 / PR 反馈
- 遇到的问题和解决方案

↓ `/reflect` (定期, 默认分析最近 10 条未处理日记)

模式识别与评估

- 频率过滤: 出现 1 次 → 忽略, 2 次 → 候选, 3+ 次 → 高置信写入
- 规则分级: Global (`~/.claude/CLAUDE.md`) vs Project-Specific (`项目/CLAUDE.md`)
- Violation Detection: 发现已有规则被违反 → 强化而非新增 (加 ZERO TOLERANCE、上移优先级)
- 去重: `processed.log` 记录已处理日记, 避免重复分析

↓ 自动写入

`CLAUDE.md` 更新

- `~/.claude/CLAUDE.md` ← 跨项目通用规则
- `[project]/CLAUDE.md` ← 项目专属规则
- `processed.log` 追加记录

23. Agent Security

AI 没有"数据"和"指令"的本质区分——凡是进入上下文窗口的文字, 都可能被当作指令执行.

攻击类型	原理
提示注入 (Prompt Injection)	恶意文本伪装成内容, 被 Agent 当作指令执行
供应链攻击	npm 包名拼写相似 (typosquat)、外部链接被篡改、社区技能隐藏延迟载荷
凭证窃取	读取 <code>.env</code> 、SSH key、API token, 通过 hook 静默外传
横向移动	Agent 有 SSH key → 渗透进生产服务器; 有多账号 → 攻陷所有渠道
MCP 工具投毒 ("Rug Pull")	你批准了某工具后, 工具描述被悄悄篡改, 指令对用户不可见但 LLM 能读取
记忆投毒	恶意片段分多次写入 Agent 记忆文件, 重启后拼合成有效攻击载荷